

```
msgid = msgget((key_t)1234, 0666 | IPC_CREAT);

if (msgid == -1) {
    printf("Error creating message queue\n");
    return 1;
}

if (fork() == 0) {
    // Child process (receiver)
    msgrcv(msgid, &message, sizeof(message.msg_text), 1, 0);
    printf("Child received: %s\n", message.msg_text);
} else {
    // Parent process (sender)
    message.msg_type = 1;
    strcpy(message.msg_text, "Hello from Parent via Message Queue!");

    msgsnd(msgid, &message, sizeof(message.msg_text), 0);
    printf("Parent sent message\n");
}

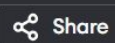
// Delete message queue
msgctl(msgid, IPC_RMID, NULL);

return 0;
}
```

▲ Parent sent message
Child received: Hello from Parent via Message Queue!

=== Code Execution Successful ===

main.c



Run

Output

Clear

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *fptr2;
5     int c;
6
7     fptr2 = fopen("output.txt", "w");
8
9     printf("Enter text (press ENTER to stop):\n");
10
11     while ((c = getchar()) != '\n') {
12         fputc(c, fptr2);
13     }
14
15     fclose(fptr2);
16
17     printf("Saved to output.txt\n");
18     return 0;
19 }
```

```
Enter text (press ENTER to stop):
hello world
Segmentation fault

=== Code Exited With Errors ===
```

```
1 #include <stdio.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <unistd.h>
5 #include <string.h>
6
7 int main() {
8     int shmid;
9     char *str;
10
11     shmid = shmget(1234, 1024, 0666 | IPC_CREAT);
12     str = (char*) shmat(shmid, NULL, 0);
13
14     if (fork() == 0) {
15         // Child reads
16         sleep(1);
17         printf("Child reads: %s\n", str);
18     } else {
19         // Parent writes
20         strcpy(str, "Hello, shared memory!");
21         printf("Data written to shared memory: %s\n", str);
22     }
23
24     shmdt(str);
25     shmctl(shmid, IPC_RMID, NULL);
26 }
```

▲ Data written to shared memory: Hello, shared memory!

=== Code Execution Successful ===

```
msgid = msgget((key_t)1234, 0666 | IPC_CREAT);

if (msgid == -1) {
    printf("Error creating message queue\n");
    return 1;
}

if (fork() == 0) {
    // Child process (receiver)
    msgrcv(msgid, &message, sizeof(message.msg_text), 1, 0);
    printf("Child received: %s\n", message.msg_text);
} else {
    // Parent process (sender)
    message.msg_type = 1;
    strcpy(message.msg_text, "Hello from Parent via Message Queue!");

    msgsnd(msgid, &message, sizeof(message.msg_text), 0);
    printf("Parent sent message\n");
}

// Delete message queue
msgctl(msgid, IPC_RMID, NULL);

return 0;
}
```

▲ Parent sent message
Child received: Hello from Parent via Message Queue!

=== Code Execution Successful ===

```

for(i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Calculate turnaround time
for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

// Display output
printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

float total_wt = 0, total_tat = 0;

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

return 0;

```

Enter Number of processes: 3

Enter Burst Time for Process 1: 1

Enter Burst Time for Process 2: 3

Enter Burst Time for Process 3: 2

Process	Burst Time	Waiting Time	Turnaround Time
P1	1	0	1
P3	2	1	3
P2	3	3	6

Average Waiting Time = 1.33

Average Turnaround Time = 3.33

=== Code Execution Successful ===

```

8     }
9     }
10    }
11    } while(!done);
12
13    // Calculate turnaround time
14    for(i = 0; i < n; i++) {
15        tat[i] = bt[i] + wt[i];
16    }
17
18    // Display results
19    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
20
21    float total_wt = 0, total_tat = 0;
22
23    for(i = 0; i < n; i++) {
24        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
25        total_wt += wt[i];
26        total_tat += tat[i];
27    }
28
29    printf("\nAverage Waiting Time = %.2f", total_wt / n);
30    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
31

```

```

Enter number of processes: 3
Enter Burst Time for Process 1:
5
Enter Burst Time for Process 2: 3
Enter Burst Time for Process 3:
1
Enter Time Quantum: 2

Process Burst Time  Waiting Time  Turnaround Time
P1  5      4      9
P2  3      5      8
P3  1      4      5

Average Waiting Time = 4.33
Average Turnaround Time = 7.33

=== Code Execution Successful ===

```

```

}

// Calculate turnaround time
for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

// Display results
printf("\nProcess\tPriority\tBurst Time\tWaiting
Time\tTurnaround Time\n");

float total_wt = 0, total_tat = 0;

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i], pr[i], bt[i], wt[i], tat[i]);

    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

return 0;
}

```

```

Enter number of processes: 3
Enter Burst Time for Process 1: 5
Enter Priority for Process 1: 2
Enter Burst Time for Process 2: 3
Enter Priority for Process 2: 1
Enter Burst Time for Process 3: 8
Enter Priority for Process 3: 3

```

	Process	Priority	Burst Time	Waiting Time	Turnaround Time
P2	1	3	0	3	
P1	2	5	3	8	
P3	3	8	8	16	

```

Average Waiting Time = 3.67
Average Turnaround Time = 9.00

```

```

=== Code Execution Successful ===

```



```

8 // Calculate waiting time
9 for(i = 1; i < n; i++) {
0     wt[i] = wt[i - 1] + bt[i - 1];
1 }
2
3 // Calculate turnaround time
4 for(i = 0; i < n; i++) {
5     tat[i] = wt[i] + bt[i];
6 }
7
8 // Display
9 printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n"
0     );
1
2 float total_wt = 0, total_tat = 0;
3
4 for(i = 0; i < n; i++) {
5     printf("P%d\t%d\t\t%d\t\t%d\n", p[i], bt[i], wt[i], tat[i]);
6     total_wt += wt[i];
7     total_tat += tat[i];
8 }
9
10 printf("\nAverage Waiting Time = %.2f", total_wt / n);
11 printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
12
13 return 0;

```

```

Enter number of processes: 3
Enter Burst Time for Process 1: 6
Enter Burst Time for Process 2: 2
Enter Burst Time for Process 3: 8

```

Process	Burst Time	Waiting Time	Turnaround Time
P2	2	0	2
P1	6	2	8
P3	8	8	16

```

Average Waiting Time = 3.33
Average Turnaround Time = 8.67

```

```

=== Code Execution Successful ===

```



```

// Calculate waiting time
for(i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Calculate turnaround time
for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

// Display results
printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

float total_wt = 0, total_tat = 0;

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t\t%d\t\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

```

```

Enter number of processes: 3
Enter Burst Time for Process 1: 5
Enter Burst Time for Process 2: 3
Enter Burst Time for Process 3: 8

```

Process	Burst Time	Waiting Time	Turnaround Time
P1	5	0	5
P2	3	5	8
P3	8	8	16

```

Average Waiting Time = 4.33
Average Turnaround Time = 9.67

```

```

=== Code Execution Successful ===

```

```
int main() {
    pid_t pid;

    // Create a new process
    pid = fork();

    if (pid < 0) {
        // Fork failed
        printf("Process creation failed!\n");
    }
    else if (pid == 0) {
        // Child process
        printf("\n--- Child Process ---\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    }
    else {
        // Parent process
        printf("\n--- Parent Process ---\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID (returned by fork): %d\n", pid);
        printf("Parent's Parent PID: %d\n", getppid());
    }

    return 0;
}
```

```
--- Child Process ---
Child PID: 15883
Parent PID: 15882
--- Parent Process ---
Parent PID: 15882
Child PID (returned by fork): 15883
Parent's Parent PID: 15873
```

```
=== Code Execution Successful ===
```

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 // Thread function 1
5 void* printNumbers(void* arg)
6 {
7     for(int i = 1; i <= 5; i++)
8     {
9         printf("Number: %d\n", i);
10    }
11    return NULL;
12 }
13
14 // Thread function 2
15 void* printLetters(void* arg)
16 {
17     for(char ch = 'A'; ch <= 'E'; ch++)
18     {
19         printf("Letter: %c\n", ch);
20     }
21     return NULL;
22 }
23
24 int main()
25 {
26     pthread t1, t2;
```

Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
Letter: A
Letter: B
Letter: C
Letter: D
Letter: E
Execution Completed

=== Code Execution Successful ===

main.c

Share

Run

```
1
2 #include <stdio.h>
3 #include <pthread.h>
4 #include <semaphore.h>
5 #include <unistd.h>
6
7 #define N 5
8
9 sem_t forks[N]; // renamed (no conflict)
10
11 // Philosopher function
12 void* philosopher(void* arg)
13 {
14     int i = *(int*)arg;
15
16     while(1)
17     {
18         printf("Philosopher %d is thinking\n", i);
19         sleep(1);
20
21         // Pick left fork
22         sem_wait(&forks[i]);
23
24         // Pick right fork
25         sem_wait(&forks[(i + 1) % N]);
26
```

Output

Clear

Philosopher 1 is thinking
Philosopher 3 is thinking
Philosopher 2 is thinking
Philosopher 0 is thinking
Philosopher 4 is thinking
Philosopher 1 is eating
Philosopher 3 is eating
Philosopher 1 is thinking
Philosopher 2 is eating
Philosopher 0 is eating
Philosopher 3 is thinking
Philosopher 2 is thinking
Philosopher 0 is thinking
Philosopher 4 is eating
Philosopher 1 is eating
Philosopher 4 is thinking
Philosopher 3 is eating
Philosopher 1 is thinking
Philosopher 0 is eating
Philosopher 3 is thinking
Philosopher 2 is eating
Philosopher 0 is thinking
Philosopher 4 is eating
Philosopher 2 is thinking
Philosopher 1 is eating
Philosopher 4 is thinking

main.c

```
1 #include <stdio.h>
2
3 #define MAX 10
4
5 // Function to display allocation
6 void printAllocation(int allocation[], int n)
7 {
8     for(int i = 0; i < n; i++)
9     {
10         if(allocation[i] != -1)
11             printf("Process %d allocated to Block %d\n", i+1,
12                 allocation[i]+1);
13         else
14             printf("Process %d not allocated\n", i+1);
15     }
16
17 // First Fit
18 void firstFit(int blockSize[], int m, int processSize[], int n)
19 {
20     int allocation[MAX];
21     for(int i = 0; i < n; i++)
22         allocation[i] = -1;
23
24     for(int i = 0; i < n; i++)
25     {
```



Run

Output

Clear

First Fit:

Process 1 allocated to Block 2
Process 2 allocated to Block 5
Process 3 allocated to Block 2
Process 4 not allocated

Best Fit:

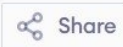
Process 1 allocated to Block 4
Process 2 allocated to Block 2
Process 3 allocated to Block 3
Process 4 allocated to Block 5

Worst Fit:

Process 1 allocated to Block 5
Process 2 allocated to Block 2
Process 3 allocated to Block 5
Process 4 not allocated

=== Code Execution Successful ===

main.c



Run

Output

Clear

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 10
5
6 char files[MAX][20];
7 int count = 0;
8
9 // Create file
10 void createFile(char name[])
11 {
12     if(count < MAX)
13     {
14         strcpy(files[count], name);
15         count++;
16         printf("File created\n");
17     }
18     else
19         printf("Directory full\n");
20 }
21
22 // Display files
23 void displayFiles()
24 {
25     printf("\nFiles in directory:\n");
26     for(int i = 0; i < count; i++)
```

File created
File created

Files in directory:
file1
file2
File found
File deleted

Files in directory:
file1

=== Code Execution Successful ===